Introducción al entorno R

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Tabla de contenidos

Introduction 3

1 Guía completa de introducción a R 3
- 1.1 Tabla de contenidos . . . 3
- 1.2 1. ¿Qué es R? . . . 3
- 1.3 2. R y la estadística: una diferencia de filosofía . . . 4
- 1.4 3. Usar R de forma interactiva . . . 4
  - 1.4.1 3.1. El prompt . . . 4
  - 1.4.2 3.2. Procedimiento recomendado al usar R por primera vez (Linux/Unix) . . . 4
  - 1.4.3 3.3. En Windows . . . 5
- 1.5 4. El sistema de ayuda de R . . . 5
  - 1.5.1 4.1. Ayuda sobre una función específica . . . 5
  - 1.5.2 4.2. Ayuda sobre operadores o palabras con significado sintáctico . . . 5
  - 1.5.3 4.3. Ayuda en formato HTML navegable . . . 5
  - 1.5.4 4.4. Búsqueda de ayuda por palabra clave . . . 5
  - 1.5.5 4.5. Ejecutar los ejemplos de un tema de ayuda . . . 5
- 1.6 5. Sintaxis básica: comandos, sensibilidad a mayúsculas, comentarios 5
  - 1.6.1 5.1. Sensibilidad a mayúsculas y minúsculas . . . 6
  - 1.6.2 5.2. Nombres válidos . . . 6
  - 1.6.3 5.3. Expresiones y asignaciones . . . 6
  - 1.6.4 5.4. Separación de comandos y agrupación . . . 6
  - 1.6.5 5.5. Comentarios . . . 6
  - 1.6.6 5.6. Límite de longitud de línea . . . 6
- 1.7 6. Asignación de variables . . . 6
- 1.8 7. Historial de comandos . . . 6
- 1.9 8. Ejecutar comandos desde un archivo y redirigir salida . . . 7
  - 1.9.1 8.1. Ejecutar un script . . . 7
  - 1.9.2 8.2. Redirigir la salida a un archivo . . . 7
- 1.10 9. Permanencia de datos y el espacio de trabajo . . . 7
  - 1.10.1 9.1. Listar los objetos actuales . . . 7
  - 1.10.2 9.2. Eliminar objetos . . . 7

Edison Achalma

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

1.10.3 9.3. Guardado del espacio de trabajo entre sesiones . 7
1.10.4 9.4. Recomendación oficial sobre directorios de trabajo 7
1.11 10. Vectores: la estructura fundamental 8
1.11.1 10.1. Crear un vector con `c()` 8
1.11.2 10.2. Asignaciones equivalentes 8
1.12 11. Aritmética vectorizada y la regla de reciclaje 8
1.13 12. Generación de secuencias . . . 8
1.13.1 12.1. El operador `:` 8
1.13.2 12.2. La función `seq()` . . 8
1.13.3 12.3. La función `rep()` . . 9
1.14 13. Vectores lógicos 9
1.15 14. Valores faltantes: `NA` y `NaN` . . 9
1.15.1 14.1. `NaN` — Not a Number 9
1.16 15. Vectores de caracteres 9
1.17 16. Vectores de índices: seleccionar y modificar subconjuntos . . . 9
1.17.1 16.1. Vector lógico 10
1.17.2 16.2. Vector de enteros positivos 10
1.17.3 16.3. Vector de enteros negativos 10
1.17.4 16.4. Vector de cadenas de caracteres 10
1.17.5 16.5. Reemplazo de elementos mediante indexación 10
1.18 17. Objetos: modo, longitud y atributos 10
1.18.1 17.1. Modo (*mode*) 10
1.18.2 17.2. Longitud (*length*) . . 10
1.18.3 17.3. Cambiar el tamaño de un objeto 11
1.18.4 17.4. Obtener y establecer atributos 11
1.19 18. Coerción entre modos 11
1.20 19. La clase de un objeto 11
1.21 20. Estructuras de control: condicionales y bucles 11
1.21.1 20.1. Condicional `if` . . . 12
1.21.2 20.2. Bucle `for` 12
1.21.3 20.3. Bucles `while` y `repeat` 12
1.22 21. Funciones propias 12
1.23 22. Paquetes: el sistema de extensión de R 12
1.23.1 22.1. Ver qué paquetes están instalados 12
1.23.2 22.2. Cargar un paquete . 12
1.23.3 22.3. Instalar y actualizar paquetes 12
1.23.4 22.4. Ver qué paquetes están cargados 12
1.23.5 22.5. Paquetes estándar (base) vs. paquetes contribuidos 13
1.24 23. Espacios de nombres (namespaces) 13
1.24.1 23.1. El operador de doble dos puntos `::` 13
1.24.2 23.2. El operador de triple dos puntos `:::` 13
1.25 24. Buenas prácticas según el manual oficial 13
1.26 25. Referencias oficiales 14

2 Publicaciones Similares 14

Introducción al entorno R

1 Guía completa de introducción a R

Referencia técnica independiente sobre los **fundamentos del lenguaje R**: qué es, cómo se usa de forma interactiva, su sintaxis básica, objetos, modos, atributos, operadores, funciones y el sistema de ayuda — todo lo previo a la manipulación de datos propiamente dicha (data frames, importación de datos, `dplyr`, etc., que se tratan en una guía aparte). Basada en el manual oficial **"An Introduction to R"** del R Core Team, publicado por CRAN (cran.r-project.org/doc/manuals), no en blogs ni tutoriales de terceros.

1.1 Tabla de contenidos

1. ¿Qué es R?
2. R y la estadística: una diferencia de filosofía
3. Usar R de forma interactiva
4. El sistema de ayuda de R
5. Sintaxis básica: comandos, sensibilidad a mayúsculas, comentarios
6. Asignación de variables
7. Historial de comandos
8. Ejecutar comandos desde un archivo y redirigir salida
9. Permanencia de datos y el espacio de trabajo
10. Vectores: la estructura fundamental
11. Aritmética vectorizada y la regla de reciclaje
12. Generación de secuencias
13. Vectores lógicos
14. Valores faltantes: `NA` y `NaN`
15. Vectores de caracteres
16. Vectores de índices: seleccionar y modificar subconjuntos
17. Objetos: modo, longitud y atributos
18. Coerción entre modos
19. La clase de un objeto
20. Estructuras de control: condicionales y bucles
21. Funciones propias
22. Paquetes: el sistema de extensión de R
23. Espacios de nombres (namespaces)
24. Buenas prácticas según el manual oficial
25. Referencias oficiales

1.2 1. ¿Qué es R?

Según la documentación oficial, R es un **conjunto integrado de herramientas de software** para manipulación de datos, cálculo y representación gráfica. Entre otras cosas, ofrece:

- una herramienta efectiva de manejo y almacenamiento de datos,
- un conjunto de operadores para cálculos sobre arreglos, en particular matrices,
- una colección amplia, coherente e integrada de herramientas intermedias para análisis de datos,
- herramientas gráficas para análisis y visualización de datos, tanto en pantalla como en copia impresa,
- un lenguaje de programación bien desarrollado, simple y efectivo (llamado **S**), que incluye condicionales, bucles, funciones recursivas definidas por el usuario, y funciones de entrada y salida.

El término "entorno" (*environment*) pretende caracterizar a R como un sistema plenamente planeado y coherente, en lugar de una acumulación incremental de herramientas muy específicas e inflexibles, como ocurre frecuentemente con otro software de análisis de datos.

R puede considerarse una implementación del lenguaje **S**, desarrollado originalmente en los Laboratorios Bell por Rick Becker, John Chambers y Allan Wilks, y que también es la base de los sistemas S-PLUS.

R es, sobre todo, un vehículo para el desarrollo de nuevos métodos de análisis de datos interactivo. Se ha extendido rápidamente mediante una gran colección de **paquetes** (*packages*). La mayoría de los programas escritos en R son, según el propio manual, esencialmente efímeros: escritos para un único análisis de datos puntual.

1.3 2. R y la estadística: una diferencia de filosofía

El manual oficial señala una diferencia importante de filosofía entre R (y S) y otros sistemas estadísticos clásicos como SAS o SPSS. En estos últimos, un análisis estadístico típicamente produce una salida abundante de forma inmediata. En S (y por tanto en R), un análisis estadístico normalmente se realiza como **una serie de pasos**, con resultados intermedios almacenados en objetos: R produce una salida mínima en cada paso y almacena los resultados en un objeto (por ejemplo, un objeto de ajuste de un modelo) para su posterior interrogación mediante otras funciones de R.

Esto tiene una implicación práctica importante para quien empieza: en R, *no ver* una salida extensa tras ajustar un modelo no significa que algo haya fallado; significa que el resultado quedó guardado en un objeto, listo para ser examinado con las funciones apropiadas (`summary()`, `print()`, etc.).

R incluye unos 25 paquetes llamados "estándar" y "recomendados" que vienen junto con cualquier instalación de R, y muchos más disponibles a través de la red de sitios CRAN (cran.r-project.org).

1.4 3. Usar R de forma interactiva

1.4.1 3.1. El prompt

Cuando se usa el programa R, este emite un *prompt* (símbolo de espera de instrucción) cuando espera comandos de entrada. El prompt por defecto es >. Si un comando no está completo al final de una línea, R emite un prompt distinto por defecto:

```
+
```

en la segunda línea y subsiguientes, y continúa leyendo hasta que el comando esté sintácticamente completo.

1.4.2 3.2. Procedimiento recomendado al usar R por primera vez (Linux/Unix)

El manual recomienda, para una primera sesión bajo UNIX/Linux, el siguiente procedimiento:

Paso 1 — Crear un subdirectorio de trabajo separado, por ejemplo llamado `work`, para contener los archivos de datos sobre los que se trabajará en ese problema concreto:

```
$ mkdir work
$ cd work
```

Paso 2 — Iniciar el programa R:

```
$ R
```

Paso 3 — A partir de aquí, pueden emitirse comandos de R.

Paso 4 — Salir del programa con el comando:

```
> q()
```

En ese momento se preguntará si se desea guardar los datos de la sesión (workspace). Según el sistema, esto aparecerá como un cuadro de diálogo o como un prompt de texto al que se puede responder `yes`, `no` o `cancel` (basta una sola letra de abreviación) para guardar los datos antes de salir, salir sin guardar, o volver a la sesión. Los datos guardados estarán disponibles en futuras sesiones de R.

Sesiones posteriores son simples: situarse en el directorio `work` e iniciar el programa de nuevo:

```
$ cd work
$ R
```

1.4.3 3.3. En Windows

El procedimiento es básicamente el mismo: se crea una carpeta como directorio de trabajo, y esta se establece en el campo "Start In" del acceso directo de R. Luego se lanza R haciendo doble clic en el ícono.

> **Nota práctica**: si usas RStudio (en cualquier sistema operativo), el equivalente moderno y recomendado a "crear una carpeta de trabajo separada" es usar **RStudio Projects** (`File → New Project...`), que gestiona automáticamente el directorio de trabajo, el historial y el workspace por proyecto.

1.5 4. El sistema de ayuda de R

R incluye un sistema de ayuda integrado, similar a la utilidad `man` de UNIX. Es, según el propio manual, la primera herramienta a dominar antes de avanzar.

1.5.1 4.1. Ayuda sobre una función específica

Para obtener información sobre una función específica, por ejemplo `solve`:

```
> help(solve)
```

Forma alternativa, más breve:

```
> ?solve
```

1.5.2 4.2. Ayuda sobre operadores o palabras con significado sintáctico

Para una característica especificada por caracteres especiales, el argumento debe ir entre comillas dobles o simples, convirtiéndolo en una "cadena de caracteres" (*character string*). Esto también es necesario para algunas palabras con significado sintáctico, incluyendo `if`, `for` y `function`:

```
> help("[[")
```

1.5.3 4.3. Ayuda en formato HTML navegable

En la mayoría de las instalaciones de R, la ayuda está disponible en formato HTML mediante:

```
> help.start()
```

Esto lanza un navegador web que permite explorar las páginas de ayuda mediante hipervínculos. El enlace "Search Engine and Keywords" en la página cargada por `help.start()` es particularmente útil, ya que contiene una lista de conceptos de alto nivel que permite buscar entre las funciones disponibles — una forma muy eficiente de orientarse rápidamente y entender la amplitud de lo que R ofrece.

1.5.4 4.4. Búsqueda de ayuda por palabra clave

El comando `help.search` (alternativamente `??`) permite buscar ayuda de varias formas:

```
> ??solve
```

Para más detalles y ejemplos, se recomienda consultar `?help.search`.

1.5.5 4.5. Ejecutar los ejemplos de un tema de ayuda

Los ejemplos incluidos en un tema de ayuda normalmente pueden ejecutarse con:

```
> example(topic)
```

1.6 5. Sintaxis básica: comandos, sensibilidad a mayúsculas, comentarios

Técnicamente, R es un **lenguaje de expresiones** (*expression language*) con una sintaxis muy simple.

1.6.1 5.1. Sensibilidad a mayúsculas y minúsculas

R **distingue mayúsculas de minúsculas** (*case sensitive*), como la mayoría de los paquetes basados en UNIX. Esto significa que `A` y `a` son símbolos distintos y se refieren a variables diferentes.

1.6.2 5.2. Nombres válidos

El conjunto de símbolos que pueden usarse en nombres de R depende del sistema operativo y del país en el que se ejecuta R (técnicamente, de la configuración regional o *locale* en uso). Normalmente se permiten todos los símbolos alfanuméricos (y en algunos países esto incluye letras acentuadas), además del punto (.) y el guion bajo (_), con la restricción de que un nombre debe empezar con un punto o una letra, y si empieza con un punto, el segundo carácter no puede ser un dígito. Los nombres son, en la práctica, ilimitados en longitud.

> **Nota de portabilidad**: para código R portable (incluido el destinado a paquetes de R), el manual recomienda usar únicamente los caracteres `A-Za-z0-9`.

1.6.3 5.3. Expresiones y asignaciones

Los comandos elementales consisten en **expresiones** o **asignaciones**:

- Si se da una expresión como comando, esta se **evalúa**, se **imprime** (a menos que se haga explícitamente invisible), y el valor se pierde.
- Una asignación también evalúa una expresión y pasa el valor a una variable, pero el resultado **no se imprime automáticamente**.

1.6.4 5.4. Separación de comandos y agrupación

Los comandos se separan mediante punto y coma (;) o mediante un salto de línea. Los comandos elementales pueden agruparse en una sola expresión compuesta mediante llaves (`{` y `}`).

1.6.5 5.5. Comentarios

Los comentarios pueden colocarse casi en cualquier parte, comenzando con el símbolo numeral (`#`); todo lo que sigue hasta el final de la línea es un comentario:

```
# esto es un comentario completo
x <- 5  # esto es un comentario al final de un
```

1.6.6 5.6. Límite de longitud de línea

Las líneas de comando introducidas en la consola están limitadas a aproximadamente 4095 bytes (no caracteres).

1.7 6. Asignación de variables

El operador de asignación estándar y tradicional en R es la flecha hacia la izquierda:

```
> x <- c(10.4, 5.6, 3.1, 6.4, 21.7)
```

Esto crea un vector llamado `x` con cinco elementos. El símbolo `<-` puede leerse como "se le asigna a", y es la forma documentada en el manual oficial para crear y nombrar objetos en R. (El operador `=` también funciona como asignación en la mayoría de los contextos, y es ampliamente usado en la práctica moderna, pero el manual de referencia introduce y emplea consistentemente `<-`).

1.8 7. Historial de comandos

Bajo muchas versiones de UNIX y en Windows, R ofrece un mecanismo para recordar y reejecutar comandos previos. Las flechas verticales del teclado permiten desplazarse hacia adelante y atrás por un **historial de comandos** (*command history*). Una vez localizado un comando de esta forma, el cursor puede moverse dentro del comando usando las flechas horizontales, y los caracteres pueden eliminarse con la tecla `DEL` o añadirse con las demás teclas.

Las capacidades de recuperación y edición bajo UNIX son altamente personalizables, mediante la biblioteca `readline`. Alternativamente, el editor de texto Emacs ofrece mecanismos de soporte más generales (vía ESS, *Emacs Speaks Statistics*) para trabajar interactivamente con R.

1.9 8. Ejecutar comandos desde un archivo y redirigir salida

1.9.1 8.1. Ejecutar un script

Si los comandos están almacenados en un archivo externo, por ejemplo `commands.R` en el directorio de trabajo `work`, pueden ejecutarse en cualquier momento de una sesión de R con el comando:

```
> source("commands.R")
```

En Windows, la opción **Source** también está disponible en el menú **File**.

1.9.2 8.2. Redirigir la salida a un archivo

La función `sink()` desvía toda la salida posterior de la consola hacia un archivo externo:

```
> sink("record.lis")
```

El comando, sin argumentos, restaura la salida a la consola nuevamente:

```
> sink()
```

1.10 9. Permanencia de datos y el espacio de trabajo

Las entidades que R crea y manipula se conocen como **objetos**. Pueden ser variables, arreglos de números, cadenas de caracteres, funciones, o estructuras más generales construidas a partir de tales componentes.

1.10.1 9.1. Listar los objetos actuales

Durante una sesión de R, los objetos se crean y almacenan por nombre. El comando:

```
> objects()
```

(alternativamente, `ls()`) muestra los nombres de (la mayoría de) los objetos actualmente almacenados en R. La colección de objetos actualmente almacenada se llama el **espacio de trabajo** (*workspace*).

1.10.2 9.2. Eliminar objetos

Para eliminar objetos está disponible la función `rm`:

```
> rm(x, y, z, ink, junk, temp, foo, bar)
```

1.10.3 9.3. Guardado del espacio de trabajo entre sesiones

Todos los objetos creados durante una sesión de R pueden almacenarse permanentemente en un archivo para uso en sesiones futuras. Al final de cada sesión de R, se ofrece la oportunidad de guardar todos los objetos actualmente disponibles. Si se indica que se desea hacerlo, los objetos se escriben a un archivo llamado `.RData` en el directorio actual, y las líneas de comando usadas en la sesión se guardan en un archivo llamado `.Rhistory`.

Cuando R se inicia más tarde desde el mismo directorio, recarga el espacio de trabajo desde ese archivo. Al mismo tiempo se recarga el historial de comandos asociado.

1.10.4 9.4. Recomendación oficial sobre directorios de trabajo

El manual recomienda explícitamente usar **directorios de trabajo separados** para distintos análisis realizados con R. Es bastante común que se creen objetos con nombres como `x` e `y` durante un análisis; estos nombres suelen ser significativos en el contexto de un único análisis, pero puede ser bastante difícil decidir qué representaban cuando se

han realizado varios análisis distintos en el mismo directorio.

> **Nota práctica**: este es, precisamente, el problema que **RStudio Projects** (y, para la gestión de paquetes por proyecto, `renv`) resuelven de forma moderna y estructurada.

1.11 10. Vectores: la estructura fundamental

R opera sobre estructuras de datos con nombre. La estructura más simple es el **vector numérico**, una entidad única que consiste en una colección ordenada de números.

1.11.1 10.1. Crear un vector con `c()`

Para configurar un vector llamado `x`, por ejemplo, consistente en cinco números (10.4, 5.6, 3.1, 6.4 y 21.7), se usa la función `c()` (*combine*):

```
> x <- c(10.4, 5.6, 3.1, 6.4, 21.7)
```

Esta es, según el manual, una **asignación de vector** usando la función `c()`, que en este contexto puede tomar un número arbitrario de argumentos vectoriales y cuyo valor resultante es un vector obtenido al concatenar sus argumentos uno tras otro.

1.11.2 10.2. Asignaciones equivalentes

El manual señala explícitamente que las siguientes formas son equivalentes a la anterior:

```
> assign("x", c(10.4, 5.6, 3.1, 6.4, 21.7)
```

y, en orden inverso (aunque poco usada en la práctica):

```
> c(10.4, 5.6, 3.1, 6.4, 21.7) -> x
```

1.12 11. Aritmética vectorizada y la regla de reciclaje

R opera sobre vectores completos sin necesidad de bucles explícitos para operaciones aritméticas básicas. Por ejemplo, si `x` es un vector ya definido, este comando crea un nuevo vector y con 11 entradas:

```
> y <- c(x, 0, x)
```

Los vectores pueden usarse en expresiones aritméticas, en cuyo caso las operaciones se realizan **elemento por elemento**. Los vectores que ocurren en la misma expresión no necesitan tener la misma longitud. Si no tienen la misma longitud, el valor del resultado es el de la expresión más larga, y los valores en el vector más corto se **reciclan** tantas veces como sea necesario (posiblemente de forma fraccionaria) hasta que coincidan en longitud con el vector más largo. En particular, una constante numérica simple es, en este sentido, un vector de longitud uno que se recicla tantas veces como sea necesario. Esta es la conocida **regla de reciclaje** (*recycling rule*).

Los operadores aritméticos elementales son los habituales `+`, `-`, `*`, `/` y `^` (potencia). Además, todas las funciones matemáticas comunes están disponibles: `log`, `exp`, `sin`, `cos`, `tan`, `sqrt`, entre otras.

1.13 12. Generación de secuencias

R provee varias formas de generar secuencias de números.

1.13.1 12.1. El operador `:`

```
> 1:30
```

genera la secuencia de enteros del 1 al 30.

1.13.2 12.2. La función `seq()`

La función `seq()` es una forma más general de generar secuencias, permitiendo especificar incrementos distintos de 1, o la longitud deseada de la secuencia en vez del valor final.

1.13.3 12.3. La función `rep()`

La función `rep()` permite generar secuencias repitiendo un vector dado un número determinado de veces.

1.14 13. Vectores lógicos

Además de las cantidades numéricas, R permite manipular **cantidades lógicas** (*logical quantities*). Los elementos de un vector lógico pueden tomar los valores `TRUE`, `FALSE`, o `NA` (valor "no disponible").

Los vectores lógicos se generan mediante condiciones. Por ejemplo:

```
> temp <- x > 13
```

establece `temp` como un vector de la misma longitud que `x`, con valores `TRUE` o `FALSE` según si el elemento correspondiente de `x` es mayor que 13 o no.

Los operadores condicionales habituales son `<`, `<=`, `>`, `>=`, `==` (igualdad exacta) y `!=` (diferente). Además, si `c1` y `c2` son expresiones lógicas, `c1 & c2` es su intersección ("y"), `c1 | c2` es su unión ("o"), y `!c1` es la negación de `c1`.

Los valores lógicos se coercionan a valores numéricos en expresiones numéricas, con `FALSE` convertido en 0 y `TRUE` convertido en 1.

1.15 14. Valores faltantes: `NA` y `NaN`

En algunos casos, los componentes de un vector pueden no estar disponibles. Cuando un elemento o valor no está disponible, se le asigna un valor particular, `NA`. En general, cualquier operación sobre un `NA` se convierte en `NA`. Esto permite que cualquier proceso computacional que reciba `NA` como argumento se propague de forma transparente a través de cálculos posteriores, dejando un resultado igualmente desconocido o faltante.

La función `is.na(x)` da un resultado lógico que indica si cada elemento de `x` es `NA` o no.

1.15.1 14.1. `NaN` — Not a Number

Hay una segunda clase de valores no disponibles producidos por cálculos numéricos, conocida como valores **`NaN`** (*Not a Number*), por ejemplo el resultado de `0/0`. Los valores `NaN` son todos `NA`, pero la afirmación inversa no es cierta. La función `is.nan(x)` se usa específicamente para detectar valores `NaN`.

1.16 15. Vectores de caracteres

Las secuencias de caracteres también son una estructura de vectores de R, generalmente utilizadas como etiquetas para componentes de un objeto. Por ejemplo:

```
> labs <- paste(c("X","Y"), 1:10, sep="")
```

crea un vector de longitud 10 mediante concatenación de pares de cadenas. La función `paste()` permite combinar cualquier número de argumentos, de manera elemento por elemento, en una sola cadena de caracteres.

Las cadenas de caracteres se escriben usando comillas dobles (preferencia indicada por el propio manual) o simples; cualquiera de las dos puede usarse para "escapar" a la otra, como en `"It's important"`.

1.17 16. Vectores de índices: seleccionar y modificar subconjuntos

Los subconjuntos de los elementos de un vector pueden seleccionarse añadiendo un **vector de índices** entre corchetes (`[]`), justo después del nombre del vector. El manual oficial distingue cuatro tipos de vectores de índices:

1.17.1 16.1. Vector lógico

El vector de índices es en sí mismo un vector lógico, de la misma longitud que el vector del que se extraen los elementos. Los valores correspondientes a `TRUE` en el vector de índices se seleccionan, y los correspondientes a `FALSE` se omiten. Por ejemplo:

```
> y <- x[!is.na(x)]
```

crea un objeto y que contiene los elementos no faltantes de `x`, en el mismo orden en que ocurren en `x`.

1.17.2 16.2. Vector de enteros positivos

El vector de índices puede consistir en un conjunto de valores enteros positivos (o vacío). En este caso, los valores del vector de índices deben estar en el conjunto `{1, 2, ..., length(x)}`. Los elementos del vector seleccionados se especifican mediante su índice, en el orden indicado.

1.17.3 16.3. Vector de enteros negativos

El vector de índices puede consistir en un conjunto de números enteros negativos. En este caso, el vector de índices especifica los valores que **se excluyen**, en lugar de los que se incluyen.

1.17.4 16.4. Vector de cadenas de caracteres

Finalmente, en el caso de un vector con nombres asociados a sus elementos (mediante el atributo `names`), el vector de índices puede ser un vector de cadenas de caracteres, refiriéndose a los elementos por nombre en lugar de por posición.

1.17.5 16.5. Reemplazo de elementos mediante indexación

Un vector indexado puede usarse en el lado izquierdo de una asignación, en cuyo caso la asignación solo opera sobre los elementos del vector indicados por el vector de índices. Por ejemplo:

```
> x[is.na(x)] <- 0
```

reemplaza todos los valores `NA` de `x` por ceros.

1.18 17. Objetos: modo, longitud y atributos

*1.18.1 17.1. Modo (*mode*)*

Las entidades sobre las que opera R se conocen técnicamente como **objetos**: ejemplos son vectores de valores numéricos (reales) o complejos, vectores de valores lógicos y vectores de cadenas de caracteres. Se conocen como estructuras "atómicas" porque todos sus componentes son del mismo tipo, o **modo**, a saber: *numeric*, *complex*, *logical*, *character* y *raw*.

Los vectores deben tener todos sus valores **del mismo modo**. Así, cualquier vector dado debe ser inequívocamente lógico, numérico, complejo, de caracteres, o raw. Una excepción aparente a esta regla es el valor especial `NA`, para cantidades no disponibles (aunque, de hecho, existen varios tipos de `NA`). Un vector puede estar vacío y aun así tener un modo: el vector vacío de caracteres se lista como `character(0)`, y el vector numérico vacío como `numeric(0)`.

R también opera sobre objetos llamados **listas** (*lists*), de modo *list*: son secuencias ordenadas de objetos que individualmente pueden ser de cualquier modo. Las listas se conocen como estructuras "recursivas" en lugar de "atómicas", porque sus componentes pueden a su vez ser listas en sí mismas.

Las otras estructuras recursivas son las de modo *function* y *expression*.

*1.18.2 17.2. Longitud (*length*)*

Otra propiedad de todo objeto es su **longitud**. Las funciones `mode(objeto)` y `length(objeto)` permiten averiguar el modo y la longitud de cualquier estructura definida. Modo y longitud se llaman, por esto, "atributos intrínsecos" de un objeto.

1.18.3 17.3. Cambiar el tamaño de un objeto

Un objeto "vacío" puede aun así tener un modo. Por ejemplo:

```
> e <- numeric()
```

hace de `e` un vector vacío de modo numérico. Una vez creado un objeto de cualquier tamaño, pueden añadírsele nuevos componentes simplemente asignándole un valor de índice fuera de su rango previo:

```
> e[3] <- 17
```

esto convierte a `e` en un vector de longitud 3 (los dos primeros componentes son, en este punto, ambos `NA`).

Inversamente, truncar el tamaño de un objeto solo requiere una asignación apropiada:

```
> alpha <- alpha[2 * 1:5]
```

o, más directamente:

```
> length(alpha) <- 3
```

1.18.4 17.4. Obtener y establecer atributos

La función `attributes(objeto)` devuelve una lista de todos los atributos no intrínsecos actualmente definidos para ese objeto. La función `attr(objeto, nombre)` permite seleccionar un atributo específico. Por ejemplo:

```
> attr(z, "dim") <- c(10,10)
```

permite a R tratar a `z` como si fuera una matriz de 10 por 10.

1.19 18. Coerción entre modos

R permite cambios de modo prácticamente en cualquier lugar donde resulte razonable hacerlo (y en algunos donde podría no serlo). Por ejemplo, con:

```
> z <- 0:9
```

se podría escribir:

```
> digits <- as.character(z)
```

tras lo cual `digits` es el vector de caracteres `c("0", "1", "2", ..., "9")`. Una coerción adicional reconstruye el vector numérico de nuevo:

```
> d <- as.integer(digits)
```

Existe una amplia colección de funciones de la forma `as.algo()` para coercionar de un modo a otro, o para dotar a un objeto de algún otro atributo que aún no posea.

1.20 19. La clase de un objeto

Todos los objetos en R tienen una **clase**, reportada por la función `class()`. Para vectores simples, esta es simplemente el modo (por ejemplo `"numeric"`, `"logical"`, `"character"` o `"list"`), pero `"matrix"`, `"array"`, `"factor"` y `"data.frame"` son otros valores posibles.

La clase es un atributo especial que permite un estilo de programación orientado a objetos en R. Por ejemplo, si un objeto tiene clase `"data.frame"`, se imprimirá de cierta forma, la función `plot()` lo mostrará gráficamente de cierta manera, y otras funciones llamadas "genéricas", como `summary()`, reaccionarán a él como argumento de una forma sensible a su clase.

Para remover temporalmente los efectos de la clase, se usa la función `unclass()`.

1.21 20. Estructuras de control: condicionales y bucles

R, como lenguaje, incluye las estructuras de control habituales de un lenguaje de programación moderno.

1.21.1 20.1. Condicional `if`

```
if (condicion) {
  expresion_verdadera
} else {
  expresion_falsa
}
```

1.21.2 20.2. Bucle `for`

```
for (nombre in vector) {
  expresion
}
```

1.21.3 20.3. Bucles `while` y `repeat`

R también incluye `while` (que repite mientras una condición sea verdadera) y `repeat` (que repite indefinidamente hasta un `break` explícito). Las palabras clave `break` y `next` permiten, respectivamente, salir de un bucle o saltar a la siguiente iteración.

1.22 21. Funciones propias

Una de las grandes fortalezas de R, según el propio manual, es la facilidad con la que el usuario puede escribir sus propias funciones. La sintaxis básica es:

```
nombre_funcion <- function(arg1, arg2, ...) {
  # cuerpo de la función
  resultado
}
```

El valor devuelto por una función es, por defecto, el valor de la última expresión evaluada en su cuerpo, aunque puede hacerse explícito con `return()`.

Las funciones en R son objetos de primera clase: pueden almacenarse en variables, pasarse como argumentos a otras funciones, y devolverse como resultado de otras funciones, igual que cualquier otro objeto de R.

1.23 22. Paquetes: el sistema de extensión de R

Todas las funciones y conjuntos de datos de R se almacenan en **paquetes** (*packages*). Solo cuando un paquete se carga, su contenido está disponible. Esto se hace tanto por eficiencia (la lista completa de funciones ocuparía más memoria y tomaría más tiempo de búsqueda que un subconjunto), como para ayudar a los desarrolladores de paquetes, que quedan protegidos de colisiones de nombres con otro código.

1.23.1 22.1. Ver qué paquetes están instalados

```
> library()
```

(sin argumentos) muestra los paquetes instalados en el sistema.

1.23.2 22.2. Cargar un paquete

Para cargar un paquete particular, por ejemplo el paquete **boot**:

```
> library(boot)
```

1.23.3 22.3. Instalar y actualizar paquetes

Los usuarios conectados a Internet pueden usar las funciones `install.packages()` y `update.packages()` (disponibles también a través del menú `Packages` en las interfaces gráficas de Windows y macOS) para instalar y actualizar paquetes.

1.23.4 22.4. Ver qué paquetes están cargados

```
> search()
```

muestra la lista de búsqueda (*search list*). Algunos paquetes pueden estar cargados pero no aparecer en la lista de búsqueda (por usar espacios de nombres); estos se incluyen en la lista entregada por:

```
> loadedNamespaces()
```

1.23.5 22.5. Paquetes estándar (base) vs. paquetes contribuidos

Los paquetes **estándar** (o *base*) se consideran parte del código fuente de R: contienen las funciones básicas que permiten que R funcione, junto con los conjuntos de datos y las funciones estadísticas y gráficas estándar descritas en el manual oficial. Deben estar disponibles automáticamente en cualquier instalación de R.

Existen, además, miles de **paquetes contribuidos**, escritos por muchos autores distintos. Algunos implementan métodos estadísticos especializados, otros dan acceso a datos o hardware, y otros están diseñados para complementar libros de texto. Algunos (los paquetes "recomendados") se distribuyen junto con toda distribución binaria de R. La mayoría están disponibles para descarga desde CRAN (cran.r-project.org y sus espejos) y otros repositorios como Bioconductor (bioconductor.org).

1.24 23. Espacios de nombres (namespaces)

Los paquetes tienen **espacios de nombres** (*namespaces*), que cumplen tres funciones:

1. Permiten al autor de un paquete ocultar funciones y datos destinados únicamente a uso interno.
2. Evitan que las funciones se rompan cuando un usuario (u otro autor de paquete) elige un nombre que colisiona con uno ya existente en el paquete.
3. Proveen una forma de referirse a un objeto dentro de un paquete particular.

1.24.1 23.1. El operador de doble dos puntos `::`

Por ejemplo, `t()` es la función de transposición en R, pero un usuario podría definir su propia función llamada `t`. Los espacios de nombres evitan que la definición del usuario tenga precedencia y rompa cualquier función que intente transponer una matriz. El operador `::` selecciona definiciones de un espacio de nombres particular: la función de transposición siempre estará disponible como `base::t`, porque está definida en el paquete `base`. Solo las funciones exportadas desde un paquete pueden recuperarse de esta forma.

1.24.2 23.2. El operador de triple dos puntos `:::`

El operador `:::` actúa como el operador `::`, pero también permite acceder a objetos ocultos. Es más probable que los usuarios recurran a la función `getAnywhere()`, que busca en múltiples paquetes.

1.25 24. Buenas prácticas según el manual oficial

A modo de síntesis, estas son recomendaciones que el propio manual de CRAN hace explícitas a lo largo del texto, antes de entrar en manipulación de datos:

- Usar **un directorio de trabajo separado por análisis**, para evitar confusión entre objetos con el mismo nombre (típicamente `x`, `y`) creados en distintos análisis no relacionados.
- Familiarizarse primero con `help()`, `?`, `help.search()` y `help.start()` antes de buscar ayuda externa: el sistema de ayuda integrado de R es, según el manual, completo y debe ser el primer recurso.
- Recordar que R **distingue mayúsculas de minúsculas** en todos los nombres.
- Aprovechar la naturaleza **vectorizada** de R: operar sobre vectores completos en lugar de escribir bucles explícitos siempre que sea razonable, ya que esto es parte del diseño fundamental del lenguaje, no solo una optimización de estilo.
- Entender la diferencia entre `NA` (valor no disponible, de cualquier tipo) y `NaN` (resultado numérico inválido específicamente, como

`0/0`), y usar `is.na()` / `is.nan()` según corresponda.
- Recordar que, en R, los resultados de un análisis estadístico normalmente se almacenan en un objeto para su examen posterior, en lugar de imprimirse exhaustivamente de inmediato.

1.26 25. Referencias oficiales

- **CRAN — "An Introduction to R" (HTML, versión release)**: https://cran.r-project.org/doc/manuals/r-release/R-intro.html
- **CRAN — "An Introduction to R" (PDF, versión release)**: https://cran.r-project.org/doc/manuals/r-release/R-intro.pdf
- **R Manuals (réplica oficial restilizada) — Índice general**: https://rstudio.github.io/r-manuals/r-intro/index.html
- **R Manuals — Cap. 1: Introduction and preliminaries**: https://rstudio.github.io/r-manuals/r-intro/Introduction-and-preliminaries.html
- **R Manuals — Cap. 2: Simple manipulations; numbers and vectors**: https://rstudio.github.io/r-manuals/r-intro/Simple-manipulations-numbers-and-vectors.html
- **R Manuals — Cap. 3: Objects, their modes and attributes**: https://rstudio.github.io/r-manuals/r-intro/Objects.html
- **R Manuals — Cap. 9: Grouping, loops and conditional execution**: https://rstudio.github.io/r-manuals/r-intro/Loops-and-conditional-execution.html
- **R Manuals — Cap. 10: Writing your own functions**: https://rstudio.github.io/r-manuals/r-intro/Writing-your-own-functions.html
- **R Manuals — Cap. 13: Packages**: https://rstudio.github.io/r-manuals/r-intro/Packages.html
- **CRAN — R FAQ (oficial)**: https://cran.r-project.org/doc/manuals/r-release/R-FAQ.html
- **CRAN — Manuales completos de R (índice)**: https://cran.r-project.org/manuals.html

---

Edison Achalma Economista — Universidad Nacional de San Cristóbal de Huamanga (UNSCH) Ayacucho, Perú ORCID: 0000-0001-6996-3364

2 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. 011 Instalacion De R
2. 012 Configurar Entorno Virtual R
3. 013 Lo Que Debemos Saber De R
4. 02 Manipulacion De Datos
5. 03 Visualizacion De Datos
6. 04 Modelo De Machine Learning I Analisis Exploratorio
7. 05 Modelo De Machine Learning Ii Modelo De Clasificacion
8. 06 Modelo De Machine Learning Iii Modelo De Regresion
9. 07 Modelo De Machine Learning Iv Tex Mining

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!